

Requisitos Funcionales

Requisitos de registro y cuentas de usuario

RF1.1 - Registrar las credenciales del usuario. El sistema debe permitir al usuario introducir unas credenciales para su registro. Para verificar la propiedad de la cuenta y evitar suplantaciones, el sistema generará un código numérico aleatorio de 6 dígitos que será enviado al correo electrónico proporcionado mediante el protocolo SMTP. El registro en la base de datos solo se completará si el usuario introduce dicho código correctamente en la interfaz. **Prioridad Alta**.

RF1.2 - Iniciar sesión en la aplicación. El sistema debe permitir al usuario emplear las credenciales usadas anteriormente para registrarse para poder iniciar sesión en la aplicación y acceder a los datos de su cuenta. **Prioridad Alta**

RF1.3 - Cerrar sesión. La aplicación debe contar con un botón de fácil acceso que permitirá al usuario cerrar la sesión activa de forma segura. **Prioridad Media**

RF1.4 - Gestionar cuenta. El sistema debe permitir acceder al usuario a una interfaz para hacer cambios en los parámetros de su cuenta. **Prioridad Baja**

RF1.5 - Cambiar contraseña. El sistema debe contar con la posibilidad de permitir al usuario cambiar la contraseña de su cuenta a través de la pantalla de gestión de cuenta. **Prioridad Baja**

RF1.6 - Eliminar la cuenta. El sistema debe permitir al usuario eliminar permanentemente su cuenta, borrando así sus datos de la base de datos a través de la pantalla de gestión de cuenta. **Prioridad Baja**

RF1.7 - Comprobar contraseña. El sistema debe comprobar si la contraseña introducida por el usuario al iniciar sesión es correcta. **Prioridad Alta**

RF1.8 - Recuperar contraseña. El usuario, en caso de olvidar su clave, debe poder introducir su correo electrónico para solicitar un restablecimiento. El sistema generará y enviará un código de verificación de 6 dígitos a dicho correo. Una vez el usuario valide este código en la aplicación, el sistema le mostrará una interfaz para establecer y guardar una nueva contraseña. **Prioridad Baja**.

Requisitos del dashboard

RF2.1 - Mostrar Dashboard. El sistema, tras haber iniciado sesión, mostrará un panel con diversos datos. **Prioridad Alta**

RF2.2 - Mostrar las estadísticas globales. El sistema debe mostrar en la pantalla principal, a través de un panel, datos tales como el número de protocolos analizados, la cantidad total de vulnerabilidades detectadas. **Prioridad Alta**

RF2.3 - Desglosar las vulnerabilidades. El sistema muestra un panel donde se debe mostrar en pantalla la cantidad de veces que se ha detectado cada tipo de fallo lógico en los protocolos del usuario. **Prioridad Media**

RF2.4 - Acceso a los protocolos recientes. La aplicación debe tener listados los últimos protocolos guardados o analizados a los que se ha accedido, permitiendo acceder a estos con un solo click. **Prioridad Media**

RF2.5 - Renombrar los protocolos guardados. El sistema, a través del panel de control, debe permitir al usuario poder renombrar los protocolos guardados en el historial. **Prioridad Baja**

RF2.6 - Eliminar los protocolos guardados. El sistema, a través del panel de control, debe permitir al usuario eliminar los protocolos guardados en el historial. **Prioridad Baja**

RF2.7 - Cargar protocolo desde el historial de protocolos recientes. El sistema debe permitir al usuario cargar en el editor los protocolos visibles en el listado de historial. El sistema deberá validar que el archivo referenciado en el path de la base de datos sigue existiendo en el disco duro local; en caso de haber sido borrado o movido por el usuario, se mostrará una advertencia y se ofrecerá la opción de eliminar el registro. **Prioridad Media**

Requisitos de archivo de protocolo

RF3.1 - Escribir los protocolos en una interfaz de notación. El sistema ha de proporcionar un área de texto donde el usuario podrá introducir la especificación del protocolo a evaluar. El sistema debe reconocer cada mensaje según un limitador como puede ser “;” o salto de línea. **Prioridad Alta**

RF3.2 - Limpiar editor de texto. El sistema da la posibilidad al usuario mediante un botón de eliminar todo el contenido presente en la interfaz de notación. **Prioridad Baja**.

RF3.3 - Numeración de líneas. El sistema debe enumerar las líneas del editor de texto para que el usuario pueda ubicar rápidamente dónde el motor de análisis ha detectado una vulnerabilidad. **Prioridad Baja**

RF3.4 - Resaltado de sintaxis básico. El área de texto deberá proporcionar diferenciación visual en las sentencias introducidas, separando estructuralmente los agentes, las flechas de dirección y el contenido encapsulado para mejorar la legibilidad del protocolo. **Prioridad Baja**.

RF4 - Definir los agentes que participan en el protocolo. El sistema debe permitir al usuario declarar los actores que participan en la comunicación. **Prioridad Alta**

RF5 - Identificar el texto cifrado en el mensaje. El sistema debe ser capaz de identificar qué parte de cada mensaje de un protocolo viaja como texto plano y qué parte encapsulada bajo alguna clave de cifrado simétrico. **Prioridad Alta**

RF6.1 - Gestionar los nonces (variable aleatorias) y marcas de tiempo en los mensajes. El sistema debe reconocer la declaración de nonces y marcas de tiempo dentro de los mensajes para evaluar correctamente el Freshness (frescura) de la comunicación. **Prioridad Alta**

RF6.2 - Cumplir Desafío-Respuesta. El sistema debe rastrear si un nonces enviado por uno de los agentes en un paso es devuelto a este (ya sea igual o con la operación esperada) por el otro agente. **Prioridad Alta**

Requisitos de Análisis y notificación de vulnerabilidades

RF7.1 - Validar la sintaxis previamente al análisis. El sistema en el momento que el usuario pulsa el botón de análisis, antes de ejecutar el motor de protocolos, debe comprobar que la sintaxis en cada una de las líneas es correcta (ej que todas las llaves de cifrado se hayan cerrado correctamente o que se respeten los agentes participantes). Si hay un error de escritura el sistema debe notificar de ello para que se solucione antes de buscar las vulnerabilidades. **Prioridad Alta**

RF7.2 - Ejecutar el motor de análisis de protocolos. El sistema, una vez se ha validado correctamente el léxico, procede a evaluar la secuencia completa de mensajes en busca de cualquier vulnerabilidad en el protocolo. **Prioridad Alta**

RF7.3 - Identificar ataques de suplantación de identidad. El sistema debe ser capaz de identificar si un protocolo es vulnerable a que un atacante suplante la identidad de uno de los agentes. **Prioridad Alta**

RF7.4 - Confirmar que el protocolo es seguro. En el caso de que el análisis de protocolos no encuentre ninguna brecha de seguridad, el sistema debe mostrar por pantalla la confirmación de que el protocolo analizado es totalmente seguro. **Prioridad Alta**

RF7.5 - Mostrar detalladamente las vulnerabilidades encontradas. Al finalizar el análisis, si ha detectado fallos, el sistema indicará los nombres de las vulnerabilidades, la línea donde se encuentra cada una y se incluirá una descripción explicándola brevemente. **Prioridad Alta**

Requisitos de importación y exportación

RF8.1 - Exportar protocolos en archivos. El sistema debe permitir exportar el protocolo que se está editando actualmente a un archivo local. **Prioridad Media**

RF8.2 - Importar protocolos desde un archivo. El sistema debe permitir cargar un protocolo desde un archivo al editor, restaurando automáticamente las líneas de mensajes. **Prioridad Media**

Requisitos de generación de informes y soporte

RF9 - Generar el informe del análisis. El sistema debe poder compilar los resultados del último protocolo analizado en un informe estructurado y permitir su exportación en formato .pdf. **Prioridad Media**

RF10 - Mostrar una guía básica de aprendizaje integrada. El sistema dispondrá de un botón de ayuda integrado que al pulsarlo desplegará una ventana con información esencial: cómo importar/exportar protocolos, la sintaxis correcta aceptada por el editor, y un resumen de funciones básicas. **Prioridad baja**

RF11 - Exigir confirmación antes de realizar acciones irreversibles. El sistema debe requerir una confirmación explícita mediante una ventana emergente antes de ejecutar acciones irreversibles (como eliminar cuenta o borrar protocolos). **Prioridad media**

Requisitos de base de datos

RF12 - Mantener persistencia del historial de análisis. El sistema debe registrar cada ejecución de análisis realizada por un usuario en la base de datos local. Para mantener la ligereza del sistema, la base de datos almacenará exclusivamente los metadatos del análisis (ID, nombre del protocolo, fecha, veredicto de seguridad y la ruta o *path* del archivo local asociado). **Prioridad Alta**

Requisitos No Funcionales

Requisito de rendimiento y robustez

RNF1 - Rendimiento del motor de análisis. La aplicación debe procesar los protocolos de forma rápida para evitar frustrar al usuario. El tiempo de ejecución desde que pulsamos el botón de analizar hasta que nos da el resultado no debe ser mayor a 4 segundos.

RNF2 - Robustez del analizador sintáctico. El sistema no debe cerrarse repentinamente si el usuario escribe mal el protocolo. Ante varias entradas de texto mal formadas, el sistema debe capturar las excepciones y mostrar un mensaje de error.

Requisitos de usabilidad

RNF3 - Accesibilidad de las funciones. El usuario no debe perder tiempo buscando donde se encuentran las funciones. Cualquier funcionalidad principal debe ser alcanzable en un máximo de 4 clicks desde la interfaz principal.

RNF4 - Cobertura de la guía integrada. La ayuda debe ser de fácil lectura. Debe explicar cómo acceder a las funcionalidades esenciales. Se entregará un cuestionario para que un grupo de usuarios valide de forma empírica este requisito de usabilidad.

Requisitos de seguridad y arquitectura

RNF5 - Cifrado de credenciales locales. Las contraseñas en la base de datos deben estar cifradas. Se considera como fallo si en la base de datos se almacena alguna contraseña en texto plano.

RNF6 - Escalabilidad de vulnerabilidades. El código debe estar diseñado para que en un futuro se añadan nuevos tipos de ataques fácilmente. Integrar nuevas comprobaciones de vulnerabilidades debe requerir únicamente modificar las clases del paquete lógico (Capa de Servicios/Motor), sin alterar la interfaz gráfica (JavaFX) ni el esquema de la base de datos. Se validará mediante inspección de código y comprobación del estricto cumplimiento del patrón MVC.